Document Number	**TP-CMPE-131 Sec xx**
Version	**1.0**
Status	**Draft**
Created By	**Rico Li**

OurSJSU

Test Plan Document

Reviewers

Department	**Name/Title**
Engineering	Evan Ha
System Architecture	Christopher Nguyen
Product Management	Taha Khan
Marketing	Min Khant Htun
Quality Assurance (QA)	Rico Li
Engineering	Kyle Ringor

Team Member Contribution Record
Project Team: After Six
Deliverable: Test Plan
Each team member must participate in the development of this deliverable. As a team, review and agree on each member's contribution level and complete the table below before submission.

Team Member	Contribution Level	Brief Description of Contribution
Evan Ha	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 7, 10
Christopher Nguyen	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Section 5, 8
Rico Li	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 3 and 9
Taha Khan	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 2.5, 5.5, 6.5, 7.5
Kyle Ringor	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Section 4 and 6
Minh Khant Htun	☐ Full (100%) ☐ Partial (50%) ☐ None (0%)	Sections 1 and 2

Contribution Levels
Full Contribution (100%) – Fully participated and contributed to the deliverable; receives 100% of the team score.
Partial Contribution (50%) – Partially participated or contributed to the deliverable; receives 50% of the team score.
No Contribution (0%) – Did not participate or contribute to the deliverable; receives 0% of the team score.

Team Confirmation:
By submitting this document, the team confirms that the contribution levels reported above have been reviewed and agreed upon by the team members.

Table of Contents

1 Introduction
    1.1 Purpose of the Document
    1.2 Scope
    1.3 Testing Goals
    1.4 Assumptions
2 Test Items
3 Testing Strategy
    3.1 Unit Testing
    3.2 Component Testing
    3.3 System Testing
    3.4 Release Testing
4 Test Environment
    4.1 Hardware
    4.2 Software
    4.3 Network
5 Test Cases
    5.1 Test Case Format
    5.2 Coverage
6 Test Schedule
    6.1 Timeline
    6.2 Milestones
7 Defect Management
    7.1 Defect Reporting
    7.2 Defect Triage
    7.3 Metrics
8 Test Deliverables
9 Risks and Mitigation
10 Appendices

List of Tables

Table 5.1

Table 6.2

Table 9

Table 10

1 Introduction

1.1 Purpose of the Document

- Define the testing objectives for OurSJSU, ensuring functional, non-functional, security, and quality standards are met prior to release.

1.2 Scope

- Functional testing (FR1–FR7), non-functional testing (baseline/peak/load testing), security checks (SQL injection, RBAC, session encryption), browser/mobile compatibility, and unit test coverage (85% backend goal).

- Direct integration with live production MySJSU backend/real student accounts (mock data used instead) and future unannounced post-v1.0 features.

1.3 Testing Goals

- Verify compliance with all SFDS requirements (FR1–FR7).
- Maintain >= 85\% backend Python code coverage and >= 80% test pass rate.
- Ensure system reliability, cross-browser compatibility and zero unresolved High/Critical defects before release.

1.4 Assumptions

- Dedicated test servers and SQLite sample databases remain available.
- Core code frozen before testing cycles.
- Mock APIs simulate external services (MySJSU login, rating endpoints).

2 Test Items

- List the components, modules, or features to be tested:
    - Module 1: User Authentication
    - Module 2: Course Data Ingestion
    - Module 3: Prerequisite Evaluation
    - Module 4: Degree Roadmap and Course Recommendations
    - Module 5: Course Search and Comparison
    - Module 6: Course Watchlist and Availability Monitoring
    - Module 7: Notification Service
    - Module 8: External Ratings Integration
    - Module 9: MySJSU Redirection

3 Testing Strategy

3.1 Unit Testing

- Test isolated methods and class logic, including password hashing, prerequisite rule evaluation algorithms (evaluate_rule), waitlist seat-drop detection, and data structures (CourseKey, StudentContext)
- Tools: Python built-in unittest / pytest framework.
- Need to maintain a minimum of 85% function/code coverage across all backend Python modules.

3.2 Component Testing

- Verify data flow between the Ingestion Web Scraper and the SQLite database
- Verify interface connections between the Prerequisite Evaluation Engine and student academic history records, and communication between the Watchlist Manager and Notification Dispatcher using simulated API responses.

3.3 System Testing

- Validate end-to-end workflows including user SSO authentication simulation, course searching and comparison, degree roadmap rendering, watchlist tracking, and notification dispatch.
    - Verify system behavior under baseline traffic, simulated peak registration spikes, and prolonged load.
    - Validate system UI and capabilities across supported web browsers and mobile responsive screens.
    - Perform penetration checks for SQL injection prevention, role-based access control, and session token encryption.

3.4 Release Testing

- Conduct full regression test suites before the Alpha Release and Beta Release milestones.
- Cross-verify executed test cases with the Requirements Traceability Matrix to ensure 100% requirement coverage and confirm zero unresolved High or Critical defects before final delivery hand-off.

4 Test Environment

4.1 Hardware

- **Minimum configuration:** A test computer with a 4-core CPU, 8 GB RAM, and 20 GB of free storage. The test server will have 2 CPU cores, 8 GB RAM, and 20 GB of storage.
- **Optimal configuration:** A test computer with an 8-core CPU, 16 GB RAM, and SSD storage. The test server will have 4 CPU cores, 16 GB RAM, and 50 GB of storage.
- Tests will use a display with a 1920 x 1080 resolution.

4.2 Software

- **Operating systems:** Windows or macOS for test computers and Linux for the test server.
- **Application and database:** Python and the OurSJSU application dependencies. A separate SQLite database will contain sample students, courses, sections, prerequisites, and watchlists.
- **Testing tools:** Python unit tests will check application logic. Chrome will be used for interface and system testing. Defects and test results will be recorded in GitHub.
- **External services:** Test data and simulated responses will be used for SJSU login. Real student accounts and records will not be used.

4.3 Network

- The test server will be separate from the production environment and accessed through HTTPS.
- A stable internet connection is required for browser tests and available external services.

5 Test Cases

5.1 Test Case Format

Test Case ID	Description	Inputs	Expected Output	Pass/Fail Criteria
TC-001	User Login Validation	Username, Password	Login successful	Correct credentials lead to successful login
TC-002	Waitlist Tracker	Course	Course added to tracker	Course is successfully added to tracker and tracked correctly
TC-003	Waitlist Notification	Course, Waitlist status, Communication method	When course waitlist clears, notification is sent to chosen platform	Notification is received on the chosen communication platform about the waitlist clearing
TC-004	Prerequisite Evaluator	Course, Course's prerequisites	Course prerequisites are correctly evaluated	Application determines the class's prerequisites are cleared when the student has them cleared, and determines they're not done when the student doesn't have them done

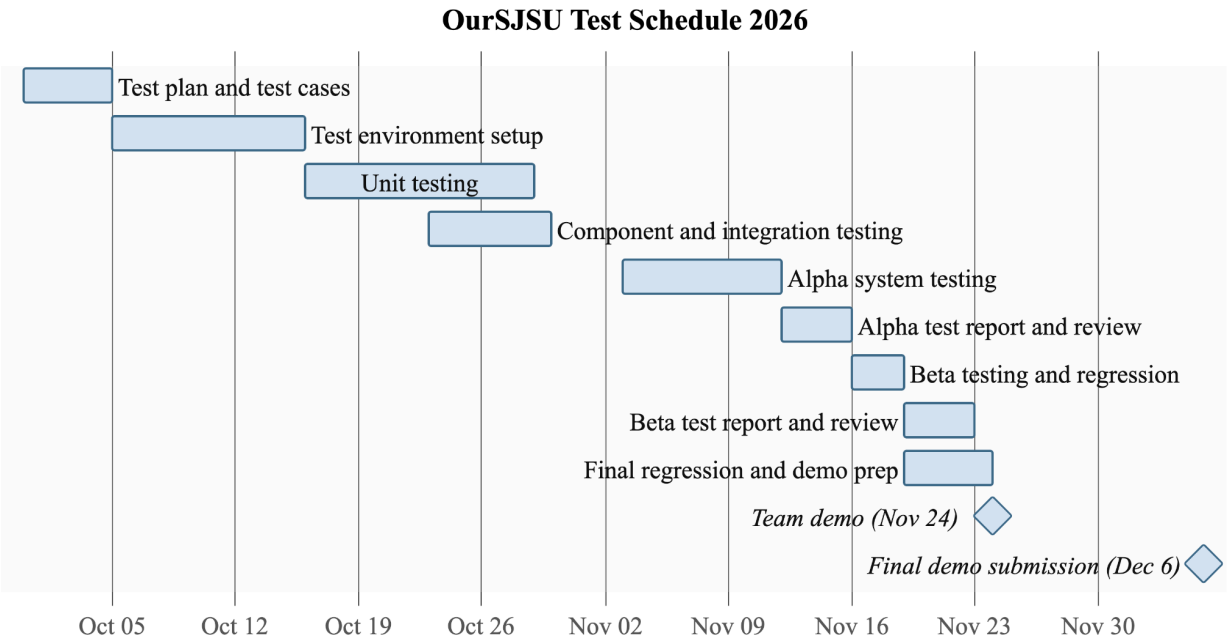
5.2 Coverage

- All functional requirements from the **Software Functional Design Specification** are covered, ranging from FR1-FR7.
- A minimum of 80% function coverage will be considered "pass" among all test cases.

- Coverage will be tested under baseline traffic, under simulated peak usage spikes, and prolonged load
- Verification of processes in different environments (different browsers, mobile devices)
- Penetration testing among different hacking methods (SQL injection, Role-based access controls)
- Inputs will be commonly used values with edge case testing->Outputs will be exactly what the function is designed to do.

6 Test Schedule

6.1 Timeline



6.2 Milestones

Milestone	Date	Description
Test Plan Submission	October 4, 2026	Finish and submit Test Plan Document.
Test Environment Ready	October 15, 2026	Finish setting up the prototype application, database, sample data, and environment for testing.

Code Review	October 27-29, 2026	In-class code review is on October 27. Formal code review meetings start October 29.
Alpha Hand-off	October 29, 2026	Provide the alpha build for validation testing.
Code Review Minutes	November 1, 2026	Submit the code review meeting minutes.
Alpha Test Report	November 15, 2026	The report and bug list will be ready for review on November 12. The report and sprint review minutes will be submitted by November 15.
Beta Test Report	November 22, 2026	The report will be ready for review on November 19. The report and sprint review minutes will be submitted by November 22.
Final Demo	November 24, 2026	Demonstrate the tested beta application.
Final Demo Submission	December 6, 2026	Submit the final demo assignment on Canvas. The tested application presentation, code, and any other necessities will be submitted.

7 Defect Management

7.1 Defect Reporting

- Defects and test results will be recorded in GitHub.
- Each defect report will include:
    - Defect ID: a unique identifier used to track the defect
    - Description: clear explanation of problem and affected feature
    - Severity: level of impact
    - Steps to Reproduce: sequence of actions to cause defect to appear
    - Status: current stage of defect in the reporting and correction process

7.2 Defect Triage

- Reported defects will be reviewed to determine the severity and priority. Defects that have a greater effect on the system functionality will be addressed before defects that have a smaller effect.
- The triage process steps:
    - Determine the severity of the defect.
    - Compare with other reported defects to determine its priority.
    - Correct defects based on their priority.
    - Review the defect report and steps to reproduce.
    - Test the affected function after correction.
    - Update status of defect based on test results.

7.3 Metrics

- Metrics that will be tracked:
    - Total number of defects identified
    - Number of defects associated with each module or feature, organized by total and severity
    - Defect density by module or feature, organized by total and severity
    - Fixed defects

8 Test Deliverables

- **Test Plan Document**: This document
- **Requirements Traceability Matrix (RTM)**: A cross-reference document mapping every functional requirement from the Software Functional Design Specification (SFDS) to its corresponding test cases to prove complete coverage.
- **Test Cases**: Complete list of test cases run
- **Test Reports**: Summaries of testing results and defect statuses
- **Defect Logs**: Records of all identified bugs defects
- **Final QA Summary & Sign-Off Report**: The ultimate closure document aggregating overall pass/fail rates, final code coverage metrics, known deferred issues, and a formal deployment recommendation.

9 Risks and Mitigation

Risk	Likelihood	Impact	Mitigation Strategy
Delay in Environment Setup	Medium	High	Allocate backup hardware/virtual resources early; use Docker containerization and pre-configured

Risk	Likelihood	Impact	Mitigation Strategy
			SQLite sample databases to rapidly deploy test environments.
Critical Bugs Before Releases	High	High	Enforce continuous integration and mandatory code reviews before merging pull requests. Prioritize high-severity defects during bug triage and block releases if critical defects remain open
Scraper Failure	Medium	High	Maintain mock HTML response files in the test suite. Implement robust error logging and parsing fallback mechanisms to allow continued testing during structure changes
Third-Party API Rate Limits and Outages	low	high	Use mock response endpoints and simulated test data during testing instead of real API calls to prevent reaching quota limits and avoid test disruptions
Tight Timeline	Medium	Medium	Strict prioritization of test cases aligned with SFDS HIGH-priority requirements. Enforce a minimum 80% coverage threshold for critical modules

10 Appendices

- Glossary:

Term	Definition

Alpha Release	An early working version provided for validation testing.
API	Application Programming Interface; allows software components to communicate.
Baseline Traffic	The normal level of system activity expected during regular use.
Beta Release	A later version evaluated before final release.
Code Coverage	The percentage of program code executed during testing.
CourseKey	A data structure used to uniquely identify a course.
Defect	A problem that causes incorrect or unexpected system behavior.
Defect Density	The number of defects associated with a module or feature.
Defect Triage	Reviewing and prioritizing reported defects based on severity.
Docker Containerization	Packaging software and its dependencies into a consistent environment.
evaluate_rule	The function used to evaluate prerequisite requirements.
GitHub	The platform used to record defects and test results.
HTTPS	A secure protocol used for network communication.
Mock Response	Simulated data used instead of a real external-service response.
Penetration Testing	Security testing that searches for system vulnerabilities.
Peak Usage	The system experiences unusual high activity.
pytest	A Python framework used to create and run tests.
Regression Testing	Retesting to ensure changes did not break existing

	functions.
Requirements Traceability Matrix (RTM)	Connects requirements to their related test cases.
Role-Based Access Control	Grants system permission based on the user's role.
Session Token	A value used to maintain an authenticated user session.
SFDS	Software Functional Design Specification
SQL Injection	An attack that submits harmful database commands through application input.
SQLite	The database system used in the test environment.
SSO	Single Sign On; the authentication method used for the SJSU login.
StudentContext	Student academic information used for the prerequisite evaluation.
Unit Testing	Separately testing individual methods, classes, and components.
unittest	A built-in framework of Python for creating and running tests.

- References:
  - Software Functional Design Specification: HERE
- Review meeting minutes (time, attendees, reviewer/review comments, action items/PICs).
  - Time: 2:15-2:45 PM
  - Attendees: Rico Li, Christopher Nguyen, Min Khant Htun, Evan Ha, Kyle Ringor, Taha Khan
  - Reviewer/Review comments:
    - TP Draft Work Division
      - Christopher - 8 & 5.5
      - Rico - 3 & 9
      - MKH - 1 & 2.5
      - Kyle 4 & 6.5
      - Evan 7.5 & 10
      - Taha 5.5, 2.5, 6.5 & 7.5
  - AI/PICs: Finish TP Draft, PICs: Rico Li, Christopher Nguyen, Min Khant Htun, Evan Ha, Kyle Ringor, Taha Khan